

lecture 10 (Network Layer -- Data plane)

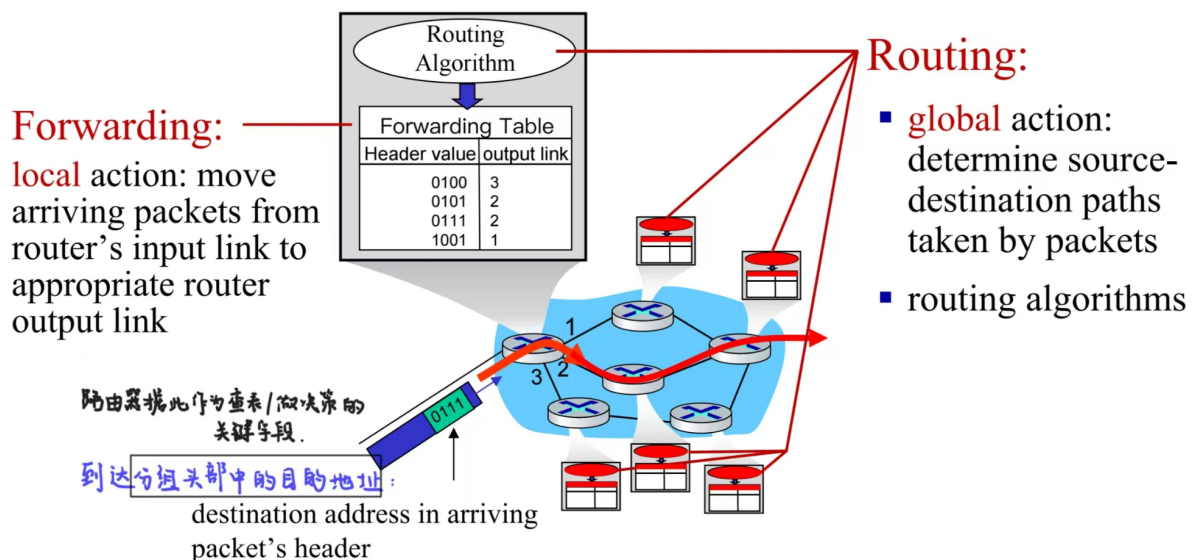
• Overview of Network Layer

• Network layer 网络层

- 网络层负责把上层（如 TCP/UDP）的数据段从源端送到目的端。
- 在发送端网络层把传输层的数据封装为 IP 数据报（datagram）并加上 IP 头。
- 接收端网络层剥去 IP 头并把上层数据交给相应的传输协议处理。
- 无论是端点主机还是中间路由器，均实现 IP 等网络层功能（但行为不同）。
- 路由器通过查看 IP 头（如目的地址、TTL、选项）来决定如何处理或转发该数据报。

• 两个关键的 network - core 功能：

•



- **转发 (Forwarding)**：局部动作——将到达的分组从路由器的输入接口移动到相应的输出接口。

- 转发是路由器按查表（forwarding table）把包送出某个端口的即时操作（数据平面）。

- **路由 (Routing)**：全局动作——决定分组从源到目的所走的路径。

- 路由涉及计算网络中端到端路径，通常由路由协议周期性协商（控制平面）。

- 路由算法：

- 实现路由决策的具体算法，例如最短路径、距离矢量、链路状态等。

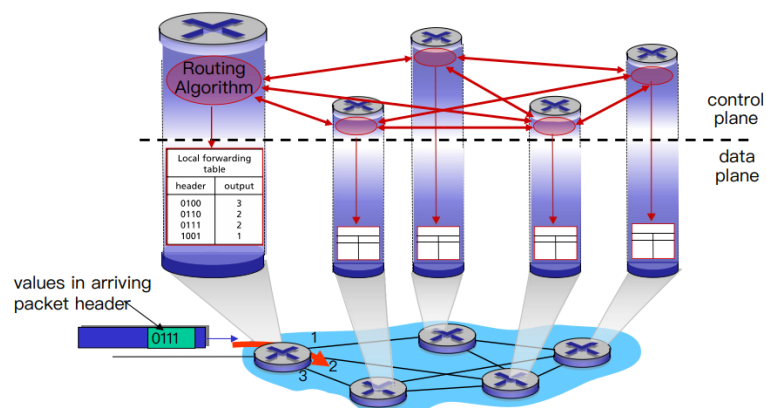
• 两个关键的 network - layer 功能：

- 网络层功能：旅行

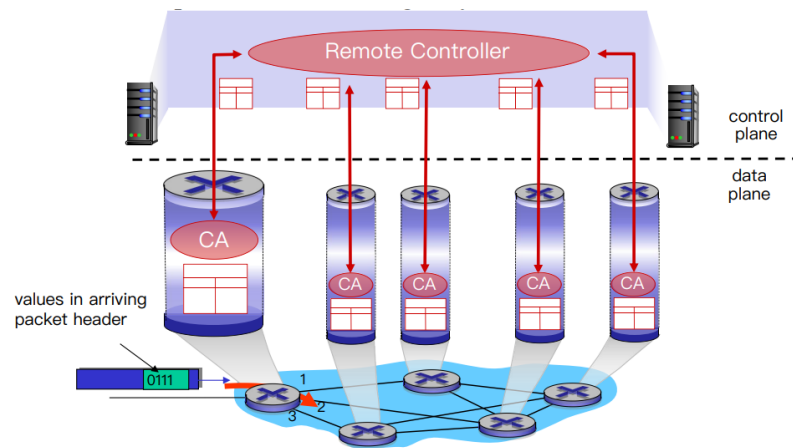
- 转发——将分组从路由器的输入端移动到相应的输出端 -- 把转发比作在路途中通过单个路口或互通立交的行为

- （属于）数据平面 Data plane

- 处理每个经过分组的本地转发动作（在路由器/硬件上完成）
- 局部的、每台路由器负责的功能，通常在硬件上实现
- 决定到达路由器输入端口的数据报如何被转发到路由器的输出端口
 - 依据转发表和包头字段做即时查表并转发
- 路由：确定分组从源到目的所走的路径 • 路由算法 -- 把路由比作规划整条路线（选路、避堵点等），决定走哪条路
 - 路由是控制平面的任务，决定转发表的内容并由路由协议维护
 - （属于）**控制平面 Control plane**
 - 负责网络级别的逻辑与决策（通常是软件/协议层面）
 - 控制平面负责收集网络信息并计算路由等策略。
 - 决定数据报在源主机到目的主机的端到端路径上如何在各路由器间被路由
 - 控制平面计算路径并下发给各路由器（或通过控制消息协商）、
 - **两种控制平面方法：**
 - **传统路由算法：**在路由器内实现
 - 每台路由器运行自己的路由协议并互相交换信息（如 OSPF、BGP）
 - 逐路由器的控制平面：每台路由器中运行的路由算法组件在控制平面上相互交互。



- 控制逻辑分散到每台路由器，靠协议消息（如邻居交换）形成全局路由信息。
- 转发与路由功能都包含在路由器内部
 - 路由器既负责**数据平面**的**转发**，也负责**控制平面**的**路由计算与协议**
- **软件定义网络（SDN）：**在（通常是远端的）服务器上实现
 - 控制逻辑集中在控制器，路由器/交换机只做转发代理
 - 逻辑上集中化的控制平面：一个独立的（通常是远端的）控制器与本地控制代理（CA）交互



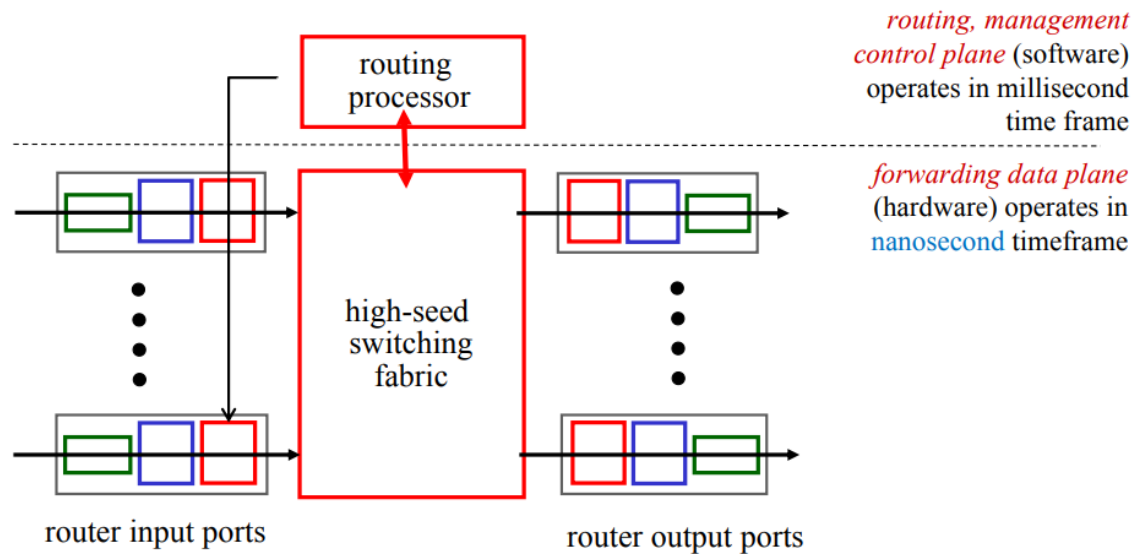
- 控制逻辑集中，路由器只保留轻量的代理来接收控制器下发的策略。
- 在 SDN 中，路由器/交换机把控制职责交给集中控制器，只专注快速的数据平面转发。
- 上方“Remote Controller”下发策略到各个 CA，数据面和控制平面分离。

- Network service model:

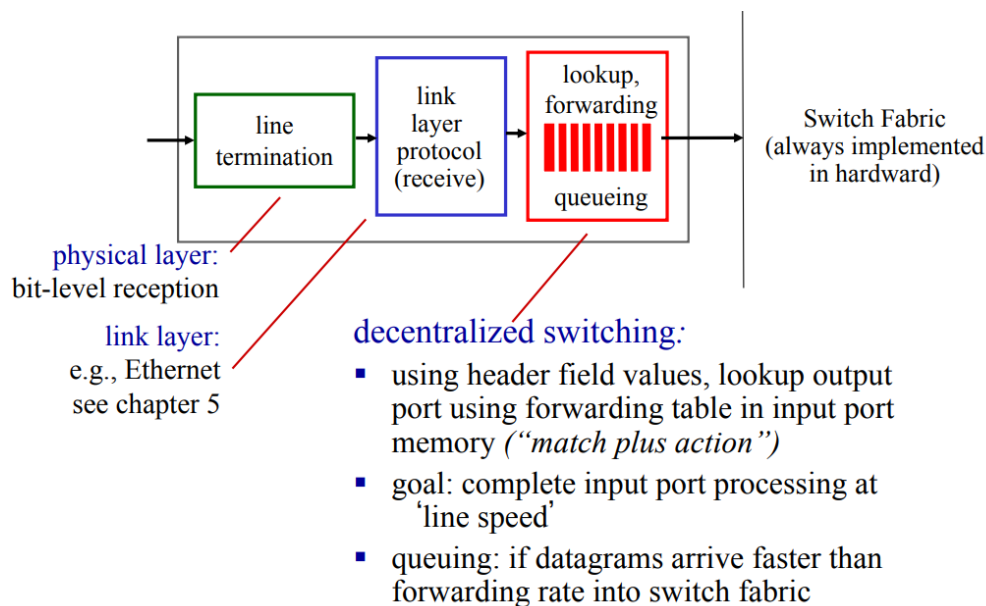
- 问：用于把数据报从发送端传到接收端的“通道”应提供什么服务模型？
- 针对单个数据报的示例服务：
 - 确保数据报最终到达目的地（无丢包）
 - 同时对延迟有严格上限的服务保证（实时要求），延迟小于 40 毫秒
- 针对一条数据流的示例服务：
 - 保证流内分组按发送顺序到达（无乱序）
 - 保留带宽资源以满足应用需求（如实时媒体）
- 其它示例服务：例如加密、认证或隔离等网络层安全保障
- 互联网的服务模型提供“尽力而为”（best-effort）服务，不对带宽、丢包、顺序或时延作出保证。
 - 公网上常见的 IP 模型没有 QoS 保证，上层需自行处理可靠性或实时性需求。

- Inside the Router

- 路由器架构概览：
 - 对通用路由器结构的高层次视图与组件介绍：



- 路由与管理控制平面（软件）：在毫秒级时间尺度内运行
 - 控制平面做的是网络级决策（如路由算法、路由表计算、网络管理），不用于逐包转发。
 - 路由处理器（routing processor）
 - 负责路由与管理逻辑（控制平面）的处理器，通常以软件运行、周期级或毫秒级更新转发表。
- 转发数据平面（硬件）：在纳秒级时间尺度内运行
 - 数据平面执行每包的查表与转发，要求极低延迟与高吞吐，通常用专用硬件实现。
 - 高速交换结构（high-speed switching fabric）
 - 路由器内部用于在输入端口和输出端口之间快速转发数据包的硬件总线/交换阵列。
- 输入端口（Input port） **做接收与初步处理和查表**
 - 路由器接收流入链路的物理/链路层并完成第一阶段处理的模块。
 -



- 物理层：比特级接收（physical layer: bit-level reception）
 - 把线上的比特流在电气/光学层面恢复成帧/符号，传给链路层。
- 链路层协议（接收）（link layer protocol (receive)）
 - 链路层对接收到的帧做错误检测、可能的重组或拆分等，然后将有效载荷交给网络层。
 - 链路层（例如以太网）：参见第5章（link layer: e.g., Ethernet; see chapter 5）
 - 负责帧边界、帧校验、MAC 地址等链路层功能；输入端口会对帧做链路层处理。
- 查表、转发、排队（lookup, forwarding, queueing）
 - 在输入端口根据数据包头查找转发表以决定目标输出端口，然后把包放入相应的队列等待进入交换结构。
 - 去中心化交换（decentralized switching）：
 - 使用报头字段值，在输入端口内存的转发表中查找输出端口（“匹配+动作”）
 - 输入端口自己做查表并决定发送到哪个输出端口（每个输入端口有自己的转发表），这就是分散式实现。
 - 集中式（输入端口只标记，再由中心交换决定）在硬件复杂性或功能扩展时有不同设计折衷。
 - 目标：在“线速”完成输入端口的全部处理
 - 要求输入端口处理速度至少与物理链路线速匹配，不成为瓶颈。
 - 排队：当数据报到达速率超过进入交换结构的转发速率时发生
 - 排队是为处理短期突发和速率不匹配而需要的缓冲机制，但会带来延迟与丢包风险。
- 基于目的地址转发（Destination-based forwarding, 转发表）
 - 转发表可以用多种方式组织（基于前缀、基于范围或基于更复杂的“match+action”规则，如 ACL、路由策略）

forwarding table

Destination Address Range	Link Interface
11001000 00010111 00010000 00000000 through 11001000 00010111 00010111 11111111	0
11001000 00010111 00011000 00000000 through 11001000 00010111 00011000 11111111	1
11001000 00010111 00011001 00000000 through 11001000 00010111 00011111 11111111	2
otherwise	3

- 用于把目的地址范围映射到出口的关键数据结构。【目的地址范围 → 链路接口】
- 左列每一行用“起始地址 through 结束地址”来表示一个地址区间（range）。当一个数据包到达时，路由器读取包头的目的地址，检查它是否在某一指定的区间内；若在该区间内，则使用该行右侧对应的接口转发该包。
 - 左列定义了“哪些目的地址属于同一下一跳/出口”的分组规则，是路由决策的匹配条件。
 - - 起始二进制：11001000 00010111 00010000 00000000 = 200.23.16.0
 - 结束二进制：11001000 00010111 00010111 11111111 = 200.23.23.255
 所以第一行表示地址区间 200.23.16.0 — 200.23.23.255，对应的下一跳接口为 0。第二行、第三行类似（例如第二行 11001000 00010111 00011000 00000000 到 11001000 00010111 00011000 11111111 即 200.23.24.0—200.23.24.255，接口 1），最后一行“otherwise”是默认路由（所有未命中前面任一范围的地址走接口 3）。
- 最后一行：otherwise → 3
 - 默认路由或兜底项，当地址不落在前面任何范围时使用默认输出。
- 问：如果这些范围不这样整齐划分（例如不同表项有重叠）会发生什么？
 - 最长前缀匹配（Longest prefix matching）
 - 查转发表时选择能匹配目的地址的“最长前缀”（最具体的条目），因为更长的前缀表示更窄更具体的地址范围，应优先使用。
 - 也可通过显式优先级（priority）或表项排序来决定，但互联网核心常用最长前缀原则（CIDR 风格）

• 交换结构（Switch fabrics） 做高速搬运

- 把包从输入端口送到正确输出端口的内部互连与交换逻辑或硬件。
- 交换速率 switching rate 表示这个【把包从输入缓冲搬到对应的输出缓冲】搬运能达到的总体吞吐
 - 1) 交换速率通常以输/出端口的链路速率倍数来表示，表示交换设备把数据从输

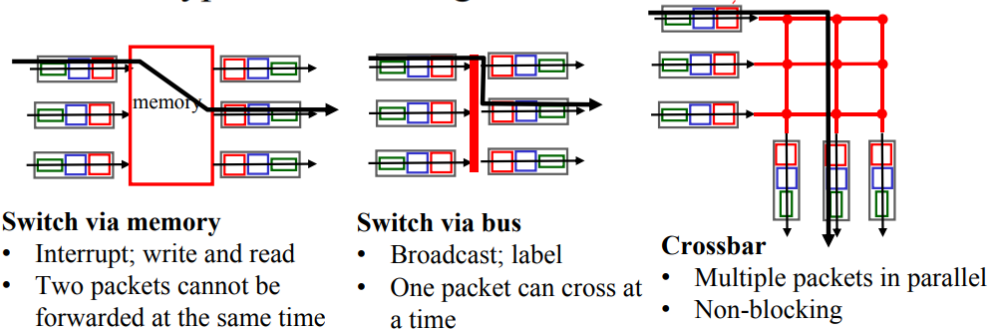
入转到输出的速率相对于单条链路速率的倍数。

- 2) 当有 N 个输入时，希望交换速率达到 N 倍链路速率，这样交换结构就能让所有 N 条输入同时以各自的全速转发，避免内部成为瓶颈（即内部阻塞）。
- 三类实现（优缺点）：

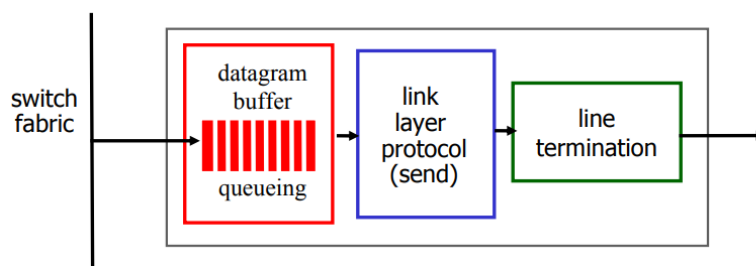
- switching rate: rate at which packets can be transferred from inputs to outputs

- often measured as multiple of input/output line rate
- N inputs: switching rate N times line rate desirable

- three types of switching fabrics

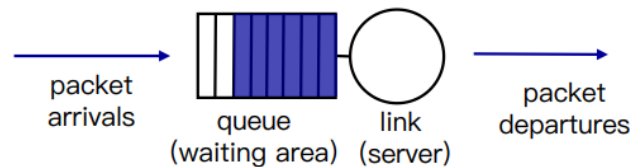


- 内存交换（写/读到共享内存，易实现但一次不能并行转发两个包）
- 总线交换（广播标签、一次只允许一包通过，总线带宽限制）
- 交叉开关（crossbar，可并行多包、非阻塞但硬件复杂成本高）
- 设计目标：在规模和成本允许下尽量选择可并行、非阻塞的交换结构以避免内部成为瓶颈；同时需要输入/输出端的队列与调度配合。
- 输出端口（Output port） 做最终调度与发送
 - 负责从路由器送出数据包到物理链路并做最终队列与调度的模块。

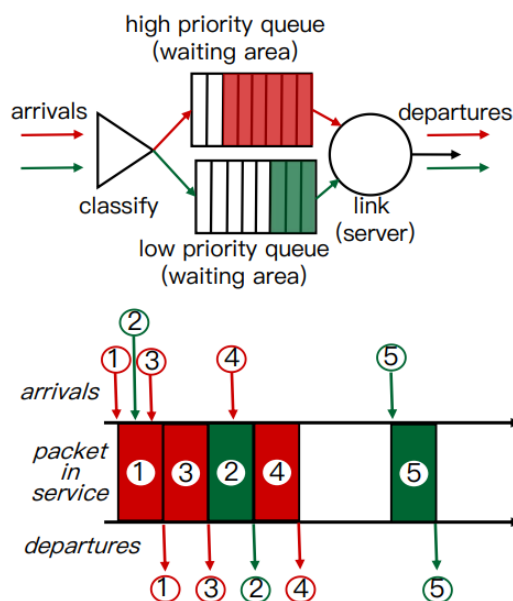


- 缓冲 buffering：
 - 输出端口在把包从交换结构取出后放入输出缓冲再由链路层发送，因此当到达速率超过发送速率时必须缓冲，否则会丢包
- 调度策略 scheduling discipline：
 - 输出端口的调度（例如优先级、轮询、公平队列）决定哪些排队的包先发，调度方案影响延迟、公平性与网络中立性（谁先得到好性能）
- 排队（Queuing）

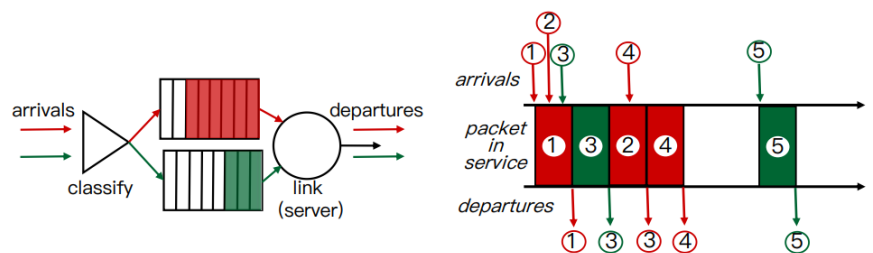
- 当到达速率超过转发速率时，包需要在输入或输出处排队等待发送。
 - 输入端口队列 (Input port queue)
 - 输出端口队列 (Output port queue)
 - 需要多少缓冲？
 - RFC 3439 的经验法则：平均缓冲量约等于“典型”RTT（比如 250 毫秒）乘以链路容量 C 。
 - 当有 N 个流时，所需缓冲量应约等于 $RTT \times C / \sqrt{N}$ 。
 - 多条独立流共享同一个瓶颈时，所需的缓冲随着流数量 N 增加而减少，按 $1/\sqrt{N}$ 缩放；当 N 很大时，每条流只需较少缓冲就能保持全利用率。
 - 调度 (Scheduling)：选择下一个在链路上发送的分组
 - 在输出端口有多个队列或候选包时，调度策略决定按何种策略（轮询、优先级、带宽保证等）决定哪个包优先发出，影响延迟、公平性与吞吐。
 - 调度策略：FIFO 先进先出



- FIFO (先进先出) 排队：按到达队列的顺序发送。
 - 容易实现，但无法区分流量优先级或服务质量。
- 丢弃策略：如果包到达时队列已满，应该丢弃谁？
 - 尾丢 (tail drop)：丢弃新到达的包。
 - 缺点是容易造成同步退避（多个 TCP 同时收敛）和不公平
 - 优先级 (priority)：按优先级丢弃/移除。

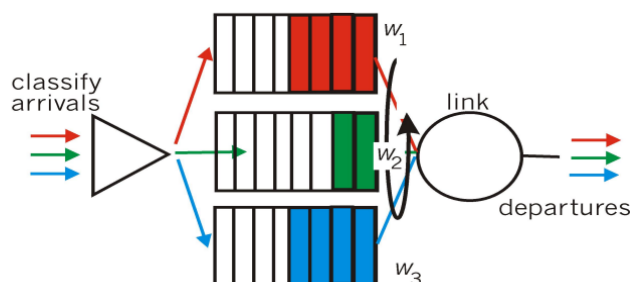


- 为某些重要流保留缓冲（例如实时语音优先），当满时先丢低优先级流的包以保护高优先级流。
- 多个类别（classes），每个类别（例如语音/视频/普通数据）有不同优先级，决定调度顺序。
 - 类别可能基于报头标记或其它信息决定，例如 IP 源/目的地、端口号等。
- 从优先级最高的类别发送一个分组。
- 非抢占式优先级排队（Non-preemptive priority queuing）。
 - 一旦一个包开始发送，不会中断它来抢先发送更高优先级的包（避免碎片化/复杂抢占），高优先只是下一个选择权。
- 随机（random）：随机丢弃/移除（例如 RED 算法的思想）。
 - 随机早期丢弃可以防止全队列突然拥塞带来的同步行为，提高整体稳定性与公平性；例如 AQM（主动队列管理）常用随机或概率丢弃。
- 轮询（Round Robin, RR）
 - 轮询把服务机会在各类别间循环分配，适合给每类一定公平机会。



- 周期性扫描各类别队列，每个类别（如有包）发送一个完整的分组。
- 调度器按顺序检查类1、类2、类3.....，每次从遇到有包的类取出并发送一个包，然后继续下一个类；这样每类都会周期性被服务，避免某一类长期饿死。
- 加权公平队列（Weighted Fair Queuing, WFQ）
 - 广义的轮询（generalized Round Robin）

Worst case: all queues have packets; $w_i / \sum_{j \in Q} w_j$



- WFQ 给每个类按权重分配服务份额，是把轮询扩展为按权重分配带宽的机制。
- 每个类别在每个周期中获得加权的服Ⓔ务量。
- 给定类 i 的带宽份额为 $w_i / \sum_{j \in Q_busy} w_j$ 。
 - w_i 是该类的权重， Q_busy 表示当前有包等待的所有类别，类 i 得到的带宽占比等于它的权重除以所有有包类别权重之和。
 - 只有当前有包的类别参与权重分母的计算，空队列的类不占用带宽份额。
- 最坏情况：所有队列都有包时，份额就是 $w_i / \sum_{j \in Q} w_j$ 。